# Engineer Documentation: Automated Forensic Analysis Tool

By: Denis Albin
Email: albindeniswork@gmail.com

This project is focused on providing a secure and efficient way to analyze hard drive images, memory dumps, and other files. It starts by carefully checking system permissions and automatically sets up the necessary forensic tools. The main features include automatic extraction and analysis, along with keeping clear and detailed logs of all actions performed during the process.

Key capabilities of the Engineer include:

Automated verification and installation of necessary forensic analysis tools.

Diverse data carving techniques.

Systematic extraction of network traffic and human-readable artifacts (e.g., passwords, usernames, URLs).

This project includes detailed memory analysis using Volatility 3. It can identify the type of memory image, list running processes, show network connections, and extract information from the system registry.

Structured reporting and persistent logging of all findings in organized directories.

The automated analysis results are automatically saved and compressed to make storage easier and sharing more efficient.

This document provides a complete overview of the Engineer, a specialized program designed for the automated analysis of hard drive and memory images and other files. Its built in functions aim to make forensic investigations simpler and more effective.

# 1. Automate HDD and Memory Analysis

## 1.1 Check the current user; exit if not ‘root’.

The script begins by ensuring it has the necessary administrative privileges to perform forensic operations.

The **root_check** function verifies that the script is executed with root permissions. This is critical because many advanced forensic tasks require elevated access to function properly. If this requirement is not met, the script will terminate, preventing incomplete or unauthorized analysis.

Example output when running the script as a non-root user:

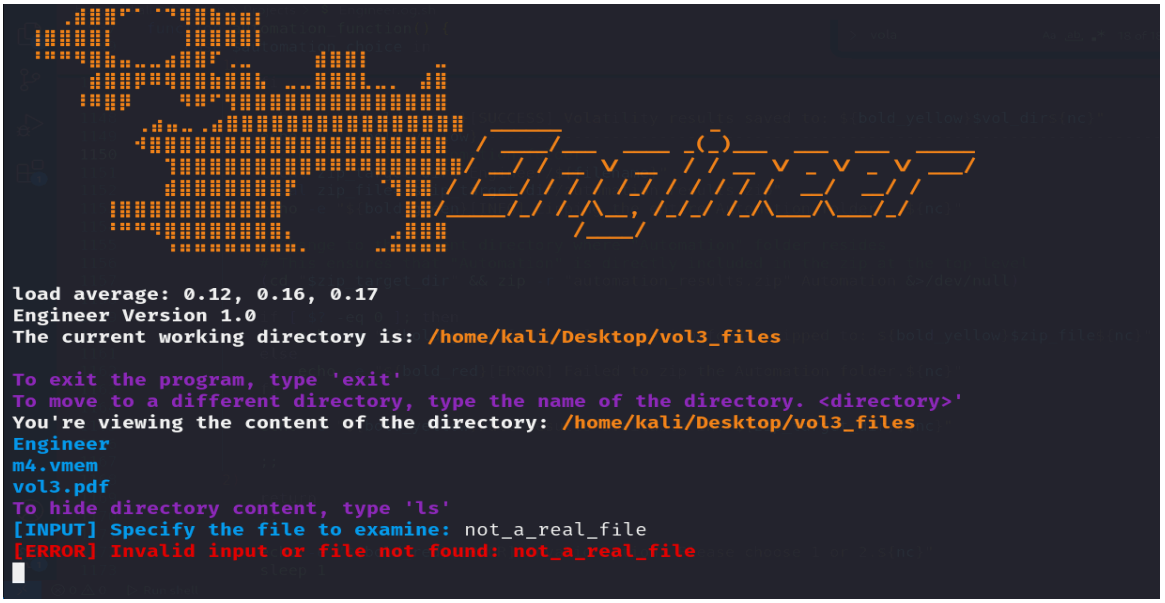



## 1.2 Allow the user to specify the filename; check if the file exists.

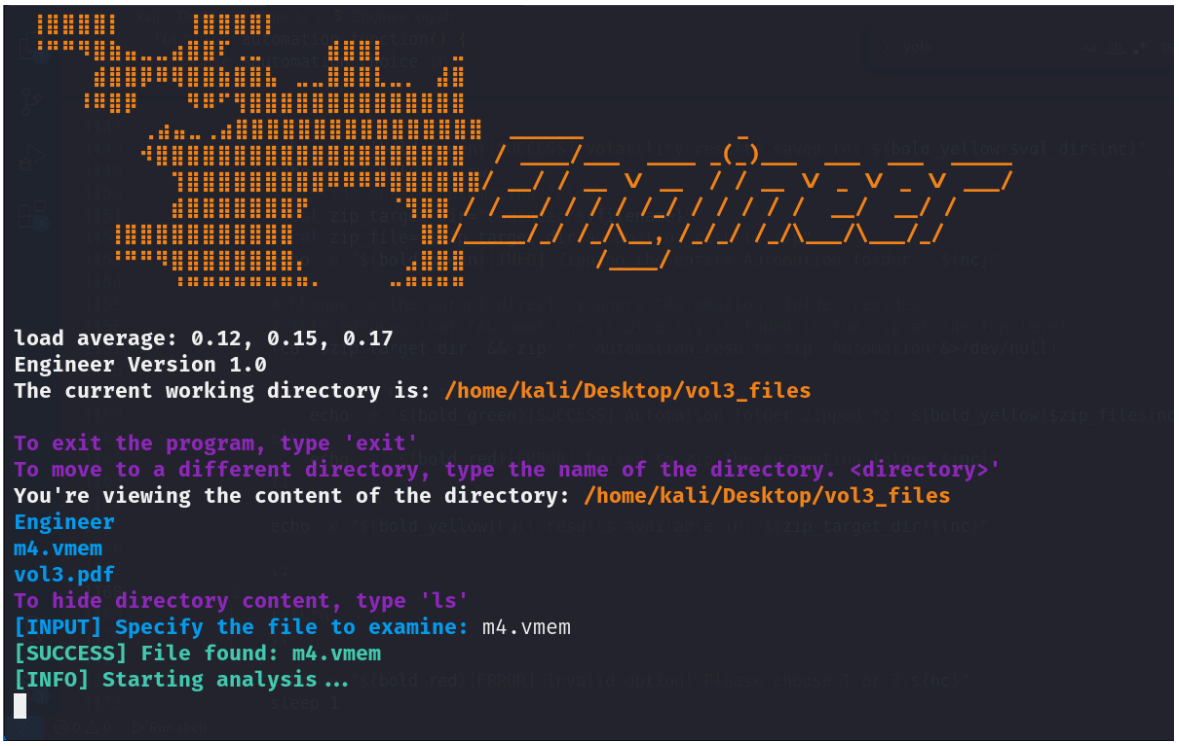
The program provides an interface for the user to select the target file for analysis and confirms its validity.

The **file_check** function guides the user in choosing the memory or HDD image file to examine. It includes a validation process to ensure the input file is valid and exists. Users can specify the file and also use commands like ls to view directory contents or cd to change directories. The option to exit is available for a clean termination of the script if needed.

Example output for if the file was not found:



Example output for successfully finding the file:

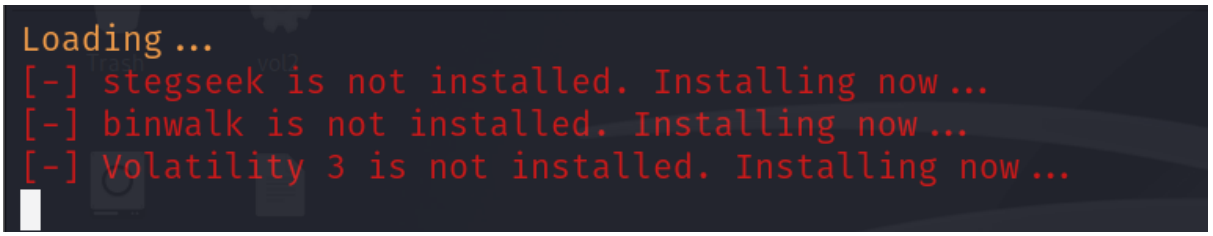



## 1.3 Create a function to install the forensics tools if missing.

Before proceeding with analysis, the script ensures that all required forensic tools are installed.

The tool_check function makes sure all the important forensic tools are available, including **bulk_extractor**, **exiftool**, **stegseek**, **foremost**, **strings**, **binwalk**, **scalpel**, **hexdump**, and **dd.** If any of these tools are missing, it automatically installs them to complete the setup for analysis.

For memory analysis, the **vol_check** function checks if **Volatility 3** is installed. If it's not found, it installs Python, pip, and then **Volatility 3**. This ensures that memory analysis features are fully ready to use.



## 1.4 Use different carvers to automatically extract data.

Engineer provides various tools for finding and checking data from forensic images, accessible through its main menu. These include **Binwalk** for listing files inside other files, **Exiftool** for viewing the file metadata, and **Bulk Extractor** for recovering other hidden items. You can use **Hexdump** to see the raw data, and **Foremost** and **Scalpel** for file carving, which means recovering lost or deleted files. **Strings** pulls out readable text, and **DD** helps get specific files using their exact locations. **Stegseek** is used to find data hidden with steganography. Additionally, the "Auto-Analysis" option runs many of these tools together to give a complete report. You can also switch the file you are analyzing or exit the program.

## 1.5 Data should be saved into a directory.

All analysis outputs and extracted data are systematically saved into an organized directory structure.

Each analysis function within Engineer automatically creates dedicated directories for its results. These directories are placed within Engineer/<filename>/<tool name>/, ensuring a structured and organized location for all collected extracted items, making them easy to access and review.

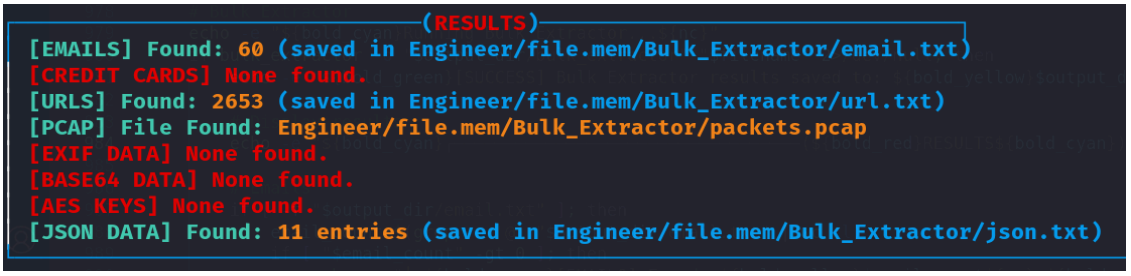
## 1.6 Attempt to extract network traffic; if found, display to the user the location and size.

The program is designed to identify and extract signs of network activity present within the forensic image, providing details to the user.

During the execution of the **bulk_extractor_menu**, the program also looks for network packet capture files, specifically those with a .pcap extension. If found, the system informs the user of its presence and exact location within the Bulk_Extractor output directory.

In the context of in-depth memory analysis, the **vol_function** (Network Connections) is set up to actively search for and show active network connections found within the memory dump. This feature helps uncover important details about network interactions and allows this information to be saved for later review.

Example of extracted pcap file:



### 1.7 Check for human-readable (exe files, passwords, usernames, etc.).

The script focuses on extracting easily readable text and potentially sensitive information that may be present within the digital files. The **strings_function** is a function designed for extracting human-readable text strings from a file. It can filter the output based on specific keywords or patterns, such as "password" or "username." The filtered information can be saved in a log file, proving effective for targeted data searches and within the **automation_function**, the strings utility is employed to automatically search for a broad range of common keywords like passwords, credentials, and network-related terms, with its output also saved to a log file for an efficient overview of potential sensitive data during automated analysis.

Example for looking for passwords:




# 2. Memory Analysis with Volatility

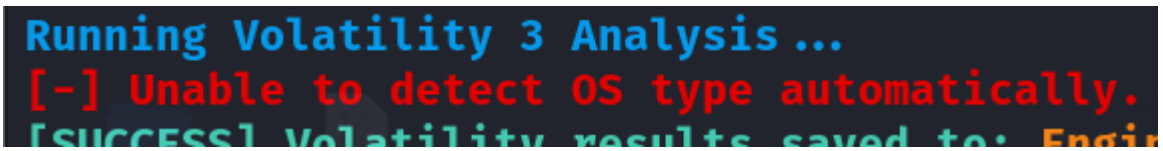
### 2.1 Check if the file can be analyzed in Volatility; if yes, run Volatility.

Before starting a full memory analysis, Engineer gives an option to confirm the validity and analyzability of the provided memory image. The **vol_function** begins by attempting to identify the operating system of the memory dump, supporting both Windows and Linux, using Volatility 3 plugins like windows.info or linux.pslist. If the OS is successfully identified, it proceeds to the Volatility analysis menu. Otherwise, the user is informed that the file might not be a valid memory dump. The **automation_function** enhances Volatility 3 integration by automatically detecting the operating system of the memory dump and running a set of OS-specific plugins. For Windows, it checks process lists, network connections, registry data, and malware indicators. For Linux, it examines process lists, bash history, socket statistics, kernel modules, and TTY activity. All results are saved in a dedicated **volatility_results** directory within the automation output.

Example for finding if the file can be analyzed with Volatility 3:




If the file can not be analyzed with Volatility 3:




## 2.2 Find the memory profile and save it into a variable.

Before starting a full memory analysis, the Engineer performs a pre-check to confirm the validity and analyzability of the provided memory image. Within the **vol_function**, choosing option 1 (Find Memory Profile) starts the script's ability to find the most suitable Volatility profile for the specific memory file This identified profile is then **saved into a variable named os_type**. Choosing the correct profile ensures the best performance and accuracy of later Volatility commands, and this important information is readily available for subsequent use or record-keeping.

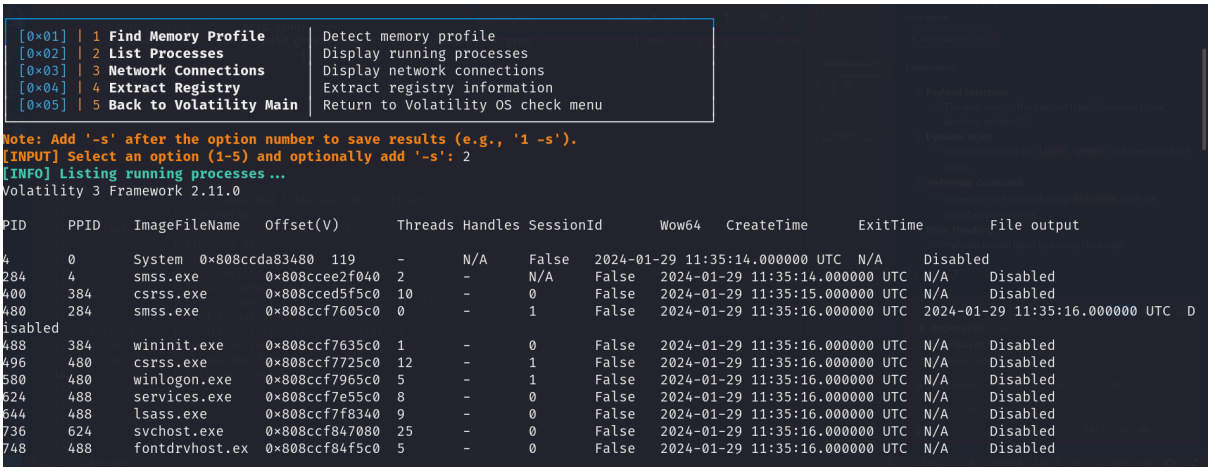
Similarly, within the **automation_function**, the script automatically identifies the operating system profile for the memory dump. This detected profile is also stored in the **os_type** variable, which then automatically guides the execution of the appropriate Volatility 3 plugins and commands ensuring the automated analysis proceeds with the correct context.

## 2.3 Display the running processes.

This function provides insight into the system's operational state by listing active processes found within the memory image.

Choosing option 2 (List Processes) within the Volatility menu tells **vol_function** to display and save a complete list of all running processes detected in the memory dump. This is done using the correct OS-specific plugin, either windows.pslist or linux.pslist. The output can be viewed directly or exported to a file for later review.

Example output:

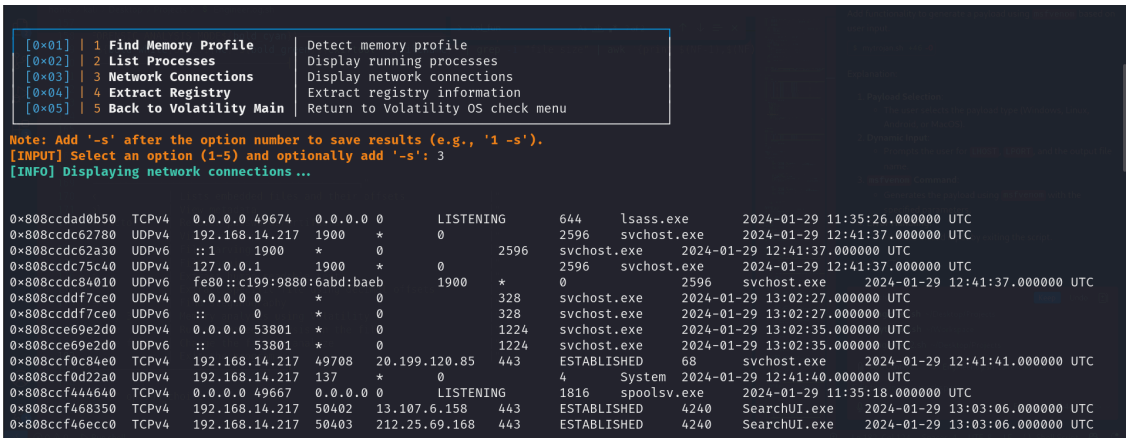



## 2.4 Display network connections.

This feature is designed to show network activity that was captured and saved within the memory image.

Through option 3 (Network Connections) within the **vol_function**, the script will display and save detailed information about any network connections found within the memory dump. This ability is very useful for understanding communication patterns and finding potentially suspicious network activities.

Example output:

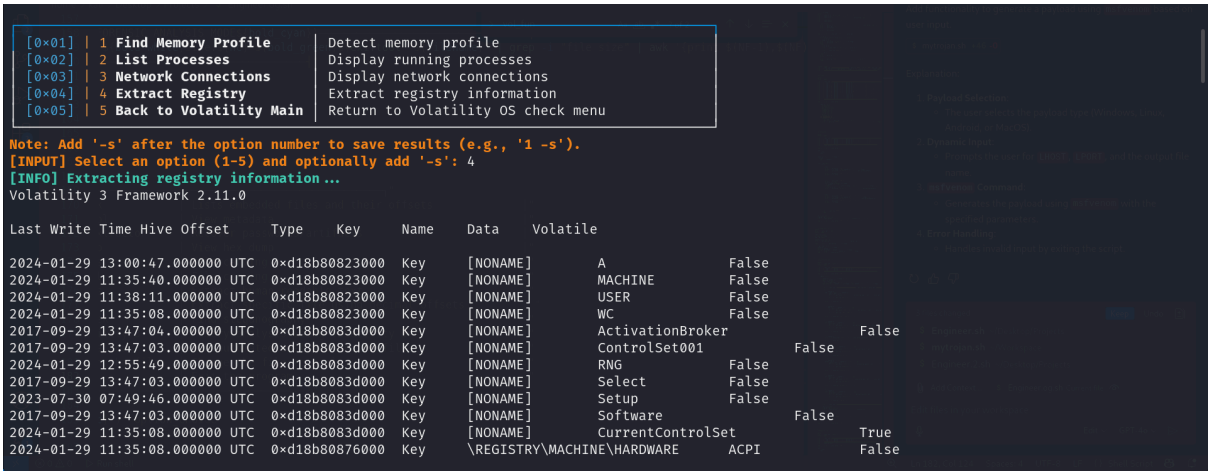


### 2.5 Attempt to extract registry information.

For memory dumps from Windows operating systems, the registry is a key source of system settings and user activity data.

Choosing option 4 (Extract Registry) within the **vol_function** starts an attempt to extract full registry information directly from Windows memory dumps, using the windows.registry.printkey plugin. It is important to note that registry analysis is a feature only for Windows operating systems; therefore, this function does not apply to Linux memory dumps. For Linux systems, other items are examined.

Example output:

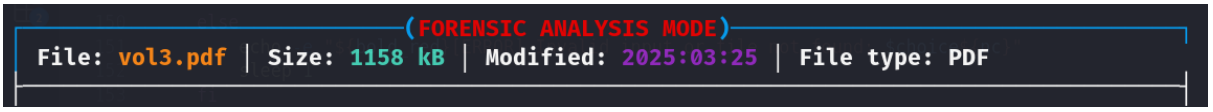



# 3. Results

### 3.1 Display general statistics (time of analysis, number of found files, etc.).

The header of the main menu, created by the **analysis_header** function, gives immediate access to initial file statistics, including file size and the last modification date.

Example:




After each specific forensic tool runs (e.g., binwalk, exiftool, bulk_extractor), a [STATS] message is created, showing the exact time taken for that particular analysis phase.

Example:

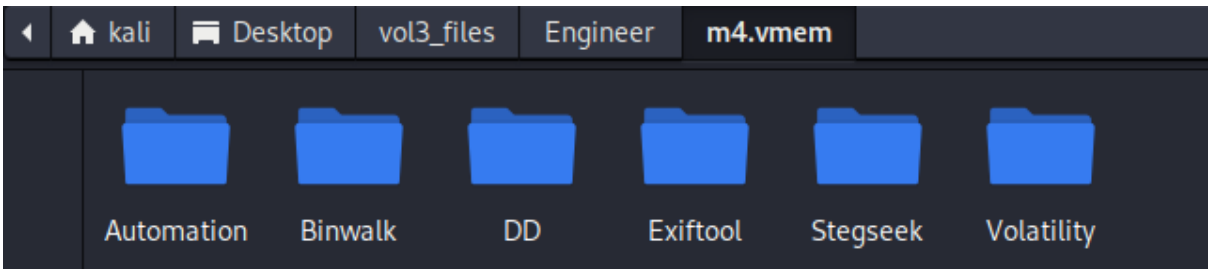


### 3.2 Save all the results into a report (name, files extracted, etc.).

Every piece of extracted data and all log files created by the script are placed into dedicated directories. These directories are nested within Engineer/<filename>/, setting up a very organized place for easy navigation and full review of all analysis results.

Each tool that runs creates its own detailed log file (e.g., quick_scan_<filename>.log, exiftool_<filename>.log). These files provide a clear, time-stamped record of every action and its result, ensuring clear and complete documentation.

Example:

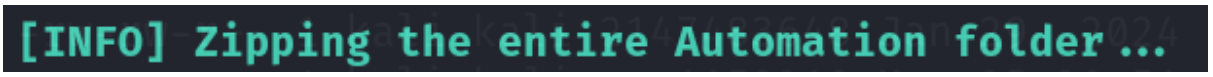


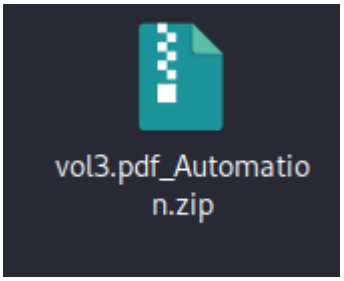

### 3.3 Zip the extracted files and the report file.

After all analysis tasks are done, Engineer.sh helps combine and package the findings for easy archiving and sharing.

After all automated tools have finished their thorough analysis, the **automation_function** proceeds to combine the entire Automation folder—which holds all individual tool outputs and logs—into a single, compressed automation_results.zip file. This process greatly improves the portability and simplifies the storage and sharing of all collected digital evidence.

Example:



# 4. Creativity

The design of Engineer focuses on being easy to use. Both the **main_menu** and its related sub-menus are set up with a clear, numbered interface. This allows for direct selection of desired options, ensuring a smooth and direct way to interact.

The use of banner designs, shown as ASCII art, helps improve the script's visual look. These elements make the user interface more engaging.

Engineer.sh uses a system of color-coding for its output messages. Green indicates successful operations, red shows errors, and yellow highlights important information. This visual difference greatly improves readability and helps quickly find important feedback.

The **analysis_header** function always shows key file details—such as the filename, size, and modification date—clearly at the top of relevant screens. This constant display ensures that the user remains fully informed about the specific file being analyzed.

Throughout the menus, especially for functions like binwalk and hexdump, brief notes are provided. These serve as helpful reminders on how to use specific features, such as saving results using the -s flag, thereby making the user more efficient.